Advanced Machine Learning

DATA 642

Lecture 4 — Ridge Regression

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, and Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier.

Shrinkage Methods

From all the norms ($p \geq 1$), the ℓ_1-norm is the only one that shows respect to small values. The rest of the ℓ_p-norms just squeeze them to make their values even smaller, and care mainly for large values.

A reason why in machine learning we use regularization is to enhance the interpretation power of an estimator.

Ridge Regression

Recall that least squares fitting procedure estimates $\theta_0, \theta_1, \theta_2, \cdots, \theta_l$ using the following cost function

$$\sum_{n=1}^{N} \left(y_i - \theta_0 - \sum_{j=1}^{l} \theta_j x_j \right)^2 \tag{1}$$

In ridge regression we actually minimize

$$\sum_{n=1}^{N} \left(y_i - \theta_0 - \sum_{j=1}^{l} \theta_j x_j \right)^2 + \lambda \sum_{j=1}^{l} \theta_j^2, \tag{2}$$

where $\lambda \geq 0$ is a hyper-parameter that controls how much one would like to regularize the model.

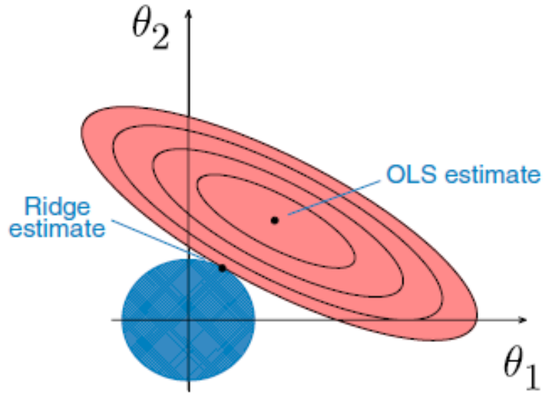
Note: The regularization term or penalty $\lambda \sum_{j=1}^{l} \theta_j^2$ forces the learning algorithm to not only fit the data but also keep the coefficient estimates as small as possible. The regularization term should only be added to the cost function during training. Once model is trained one would use the unregularized performance measure to evaluate model's performance.

1. When $\lambda = 0$, ridge regression becomes LSE.
2. When $\lambda \to \infty$, the ridge regression coefficients will approach zero.

It is worth to mention that penalty is applied only to $\theta_1, \theta_2, \cdots, \theta_l$ and not to θ_0. So if all columns of data matrix $\mathbf{X} \in \mathbb{R}^{N \times l}$ are centered then

$$\hat{\theta}_0 = \bar{y} = \sum_{n=1}^{N} y_n / N \tag{3}$$

Geometric interpretation



From an optimization perspective in the special case where $l = 2$, the constraint in ridge regression corresponds to a circle,

$$\theta_1^2 + \theta_2^2 \leq t, \tag{4}$$

where $\sqrt{t}$ is the radius of the circle. In this task, we are trying to minimize the ellipse size and circle simultaneously in the ridge regression. The ridge estimate is given by the point at which the ellipse and the circle touch.

1. When $\lambda = 0$ we are trying to minimize the ellipse size. The inner ellipse has smaller RSS. When features are highly correlated, optimization becomes non-trivial.
2. In terms of the bias-variance trade-off smaller λ provide more variance and less bias. Larger λ provide some bias but less variance.

The solution

Given the objective function for ridge regression:

$$J(\boldsymbol{\theta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \lambda \boldsymbol{\theta}^\top \boldsymbol{\theta} \tag{5}$$

Where:

- $\mathbf{y}$ is the vector of target values.
- $\mathbf{X}$ is the design matrix.
- $\boldsymbol{\theta}$ is the vector of coefficients.
- λ is the regularization parameter.

To find the minimum of $J(\boldsymbol{\theta})$, we take the derivative with respect to $\boldsymbol{\theta}$ and set it equal to zero:

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = 0 \tag{6}$$

Expanding and simplifying the objective function:

$$\begin{aligned} J(\boldsymbol{\theta}) &= (\mathbf{y}^\top - \boldsymbol{\theta}^\top \mathbf{X}^\top)(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \lambda \boldsymbol{\theta}^\top \boldsymbol{\theta} \\ &= \mathbf{y}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} + \lambda \boldsymbol{\theta}^\top \boldsymbol{\theta} \end{aligned}$$

Now, taking the derivative with respect to $\boldsymbol{\theta}$ and setting it equal to zero:

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} + 2\lambda \boldsymbol{\theta} = 0 \tag{7}$$

Solving for $\boldsymbol{\theta}$, we get:

$$\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} + \lambda\boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y} \tag{8}$$

Factoring out $\boldsymbol{\theta}$, we obtain:

$$(\mathbf{X}^\top \mathbf{X} + \lambda\mathbf{I})\boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y} \tag{9}$$

Finally, solving for $\boldsymbol{\theta}$, we get the closed-form solution for ridge regression:

$$\hat{\boldsymbol{\theta}}_R = (\mathbf{X}^\top \mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top \mathbf{y} \tag{10}$$

This is the desired closed-form solution for ridge regression derived using matrix notation.

Comments

1. **Accurate Predictions and Reduced Variance**: Ridge regression provides accurate predictions and reduces variance.

2. **Controlled Overfitting**: Ridge regression effectively controls overfitting by penalizing the magnitude of the coefficients. This prevents the model from learning intricate details of the training data that may not generalize well to unseen data, leading to more robust predictions.

3. **Handles Multicollinearity**: Ridge regression is particularly useful when dealing with multicollinearity, where predictor variables are highly correlated with each other. By shrinking the coefficients, ridge regression mitigates the issue of multicollinearity and stabilizes the parameter estimates, improving the numerical stability of the model.

Computational complexity

1. As with linear regression, we can perform ridge regression either by computing a closed-form equation or by performing gradient descent.
2. From the ridge regression closed form solution we see that in order to obtain a solution we need to compute the inverse of a matrix. Although there are sophisticated matrix factorization techniques for this (Cholesky), sometimes this approaches get very slow when the number of features grows large.
3. In the case where there are a large number of features/variables or too many training instances to fit in memory it may be better to use gradient descent.